

DSC 291: Learning Theory Assignment 6

Tongle Shen

May 18, 2026

1 Part A: Boosting Sparse Linear Predictors and the ℓ_1 Margin

1.1 Problem 1. A Coordinate Weak Learner from Sparse Margin

Fix any distribution Q supported on examples satisfying

$$y\langle w^*, \phi(x) \rangle \geq 1, \quad \|w^*\|_0 \leq s.$$

Write

$$\mu_j := \mathbb{E}_Q[y \phi_j(x)].$$

Taking expectation in the margin inequality gives

$$1 \leq \mathbb{E}_Q \left[y \sum_{j=1}^d w_j^* \phi_j(x) \right] = \sum_{j=1}^d w_j^* \mu_j.$$

Therefore

$$1 \leq \sum_{j=1}^d |w_j^*| |\mu_j| \leq \|w^*\|_1 \max_{j \in [d]} |\mu_j|.$$

Hence there exists some coordinate j^* such that

$$|\mu_{j^*}| \geq \frac{1}{\|w^*\|_1}.$$

Now choose

$$\sigma^* := \text{sign}(\mu_{j^*}), \quad b(x) := b_{j^*, \sigma^*}(x) = \sigma^* \phi_{j^*}(x).$$

Then

$$\mathbb{E}_Q[y b(x)] = \sigma^* \mathbb{E}_Q[y \phi_{j^*}(x)] = |\mu_{j^*}| \geq \frac{1}{\|w^*\|_1}.$$

Since

$$\frac{1}{2} - L_Q(b) = \frac{\mathbb{E}_Q[y b(x)]}{2},$$

we obtain

$$L_Q(b) \leq \frac{1}{2} - \frac{1}{2\|w^*\|_1}.$$

Because w^* is s -sparse and $\|w^*\|_\infty \leq B$,

$$\|w^*\|_1 \leq sB,$$

so also

$$L_Q(b) \leq \frac{1}{2} - \frac{1}{2sB}.$$

1.1.1 Weighted ERM over $\mathcal{B}$

Given weighted examples

$$(x_i, y_i, D_i)_{i=1}^n, \quad D_i \geq 0, \quad \sum_{i=1}^n D_i = 1,$$

the weighted error of $b_{j,\sigma}$ is

$$L_D(b_{j,\sigma}) = \sum_{i=1}^n D_i \mathbf{1}[b_{j,\sigma}(x_i) \neq y_i] = \frac{1}{2} - \frac{1}{2} \sum_{i=1}^n D_i y_i \sigma \phi_j(x_i).$$

For each coordinate define the weighted correlation

$$c_j := \sum_{i=1}^n D_i y_i \phi_j(x_i).$$

For fixed j , the better sign is $\sigma = \text{sign}(c_j)$, giving error

$$\frac{1}{2} - \frac{|c_j|}{2}.$$

So weighted ERM over $\mathcal{B}$ is:

- compute all c_j for $j \in [d]$,
- choose $j^* \in \arg \max_j |c_j|$,
- output $b_{j^*, \text{sign}(c_{j^*})}$.

Computing every c_j requires one pass over the weighted sample and costs $O(nd)$ time.

1.2 Problem 2. AdaBoost and the Generalization Bound

Run AdaBoost on the base class $\mathcal{B}$. By Problem 1, every reweighted distribution encountered by AdaBoost admits a weak learner with error at most

$$\varepsilon_t \leq \frac{1}{2} - \gamma, \quad \gamma := \frac{1}{2sB}.$$

AdaBoost chooses

$$\alpha_t = \frac{1}{2} \log \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right),$$

and its standard normalization factor is

$$Z_t = 2\sqrt{\varepsilon_t(1 - \varepsilon_t)}.$$

Therefore the empirical training error after T rounds satisfies

$$\hat{L}(H_T) \leq \prod_{t=1}^T Z_t \leq \left(2\sqrt{\left(\frac{1}{2} - \gamma\right)\left(\frac{1}{2} + \gamma\right)} \right)^T = \left(\sqrt{1 - 4\gamma^2} \right)^T.$$

Using $\sqrt{1 - u} \leq e^{-u/2}$,

$$\hat{L}(H_T) \leq e^{-2\gamma^2 T}.$$

Substituting $\gamma = 1/(2sB)$ gives

$$\hat{L}(H_T) \leq \exp \left(-\frac{T}{2s^2 B^2} \right).$$

To force zero training error on an n -point sample, it is enough to make the right-hand side smaller than $1/n$. Thus it suffices to choose

$$T \geq 2s^2 B^2 \log n.$$

1.2.1 From Boosting to a Sparse Linear Classifier

Each weak learner has the form

$$h_t(x) = \sigma_t \phi_{j_t}(x), \quad \sigma_t \in \{-1, +1\}.$$

Hence the boosted score can be written as

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x) = \left\langle \sum_{t=1}^T \alpha_t \sigma_t e_{j_t}, \phi(x) \right\rangle.$$

So the final classifier is

$$H_T(x) = \text{sign}(F_T(x)) = \text{sign}(\langle w_T, \phi(x) \rangle)$$

for

$$w_T := \sum_{t=1}^T \alpha_t \sigma_t e_{j_t}.$$

Its support satisfies

$$|\text{supp}(w_T)| \leq T.$$

Now apply the sparse-linear VC bound with activated-coordinate budget T :

$$\text{VCdim}(\mathcal{H}_T) = O\left(T \log\left(\frac{ed}{T}\right)\right),$$

where

$$\mathcal{H}_T := \{x \mapsto \text{sign}(\langle w, \phi(x) \rangle) : \|w\|_0 \leq T\}.$$

Since AdaBoost produces a sample-consistent hypothesis once $T \geq 2s^2 B^2 \log n$, the realizable VC theorem gives true error at most ε with probability at least $1 - \delta$ provided

$$n = O\left(\frac{T \log(ed/T) + \log(1/\delta)}{\varepsilon}\right).$$

Substituting $T = O(s^2 B^2 \log n)$ and absorbing the extra logarithms into $\tilde{O}(\cdot)$ yields

$$n = \tilde{O}\left(\frac{s^2 B^2 \log d + \log(1/\delta)}{\varepsilon}\right).$$

1.2.2 Comparison with Exact Sparse ERM

Exact sparse ERM has realizable sample complexity

$$\tilde{O}\left(\frac{s \log(ed/s) + \log(1/\delta)}{\varepsilon}\right),$$

so boosting pays a statistical price of roughly an extra factor proportional to

$$\frac{1}{\gamma^2} = s^2 B^2.$$

This is the cost of replacing exact combinatorial search by a weak-learning reduction.

The computational advantage is that each boosting round only needs the $O(nd)$ weighted ERM from Problem 1, so the total training time is

$$O(ndT) = O(nd s^2 B^2 \log n).$$

This avoids the hard support-search subproblem of exact sparse ERM.

The parameter B is essential. If the realizing separator has very uneven coefficients, then $\|w^*\|_1$ can be much larger than s , which shrinks the weak edge and increases both the number of rounds and the final generalization complexity.

1.3 Problem 3. A Hard Construction Showing Why B Matters

Let

$$s = 2m + 1 \quad \text{and} \quad d = s.$$

Index the s support points by

$$0, (1, +), (1, -), \dots, (m, +), (m, -),$$

and let every label be $y = +1$.

We construct s coordinates, grouped as one base column, m flip columns, and m carry columns.

1.3.1 The Feature Matrix

Let the columns be

$$c_0, f_1, \dots, f_m, g_1, \dots, g_m \in \{-1, +1\}^s.$$

They are defined row-wise as follows.

Base column c_0 .

$$c_0(0) = +1, \quad c_0(r, +) = +1, \quad c_0(r, -) = -1.$$

Flip column f_t . It matches c_0 except on the pair $(t, +), (t, -)$, where the signs are flipped:

$$f_t(0) = +1, \quad f_t(t, +) = -1, \quad f_t(t, -) = +1,$$

and for $r \neq t$,

$$f_t(r, +) = +1, \quad f_t(r, -) = -1.$$

Carry column g_t . Set

$$g_t(0) = -1.$$

For a pair indexed by r :

- if $r < t$, then $g_t(r, +) = g_t(r, -) = -1$;
- if $r = t$, then $g_t(t, +) = g_t(t, -) = +1$;
- if $r > t$, then $g_t(r, +) = +1$ and $g_t(r, -) = -1$.

Let $M \in \{-1, +1\}^{s \times s}$ be the matrix whose columns are

$$M = [c_0 \ f_1 \ \dots \ f_m \ g_1 \ \dots \ g_m].$$

1.3.2 Every Coordinate Has Exponentially Small Edge

Assign weights

$$q_0 = 1, \quad q_{r,+} = q_{r,-} = 2^{r-1} \quad \text{for } r \in [m].$$

Let

$$Z := q_0 + \sum_{r=1}^m (q_{r,+} + q_{r,-}) = 2^{m+1} - 1,$$

and define a distribution Q on the s support points by probability q_ρ/Z on row ρ .

By direct inspection, every column of M has weighted sum exactly 1. For the three column types,

$$\sum_{\rho} q_{\rho} c_0(\rho) = 1 + \sum_{r=1}^m 2^{r-1} (1 - 1) = 1,$$

$$\sum_{\rho} q_{\rho} f_t(\rho) = 1 + \sum_{r \neq t} 2^{r-1} (1 - 1) + 2^{t-1} (-1 + 1) = 1,$$

and

$$\sum_{\rho} q_{\rho} g_t(\rho) = -1 + \sum_{r < t} 2^{r-1} (-1 - 1) + 2^{t-1} (1 + 1) + \sum_{r > t} 2^{r-1} (1 - 1)$$

$$= -1 - \sum_{r < t} 2^r + 2^t = -1 - (2^t - 2) + 2^t = 1.$$

Hence

$$\sum_{\rho} q_{\rho} M_{\rho, j} = 1 \quad \text{for every column } j.$$

Since all labels are +1, each coordinate literal has correlation

$$\mathbb{E}_Q[\phi_j(x)] = \frac{1}{Z},$$

or its negative if we flip the sign. Therefore the best literal based on coordinate j has edge exactly

$$\frac{1}{2Z} = \frac{1}{2(2^{m+1} - 1)} = 2^{-\Theta(m)} = 2^{-\Omega(s)}.$$

So every coordinate predictor has exponentially small edge.

1.3.3 An Explicit Margin-1 Separator

Define coefficients

$$a_0 := (2 - m)2^m - 1, \quad a_t := 2^m - 2^{m-t}, \quad b_t := 2^{m-t} \quad \text{for } t \in [m].$$

Let

$$w^* := (a_0, a_1, \dots, a_m, b_1, \dots, b_m) \in \mathbb{R}^s.$$

We claim that every row of M has inner product exactly 1 with w^* .

Let

$$A := \sum_{t=1}^m a_t = (m-1)2^m + 1, \quad B := \sum_{t=1}^m b_t = 2^m - 1.$$

Then

$$a_0 + A = 2^m.$$

Row 0. Since row 0 equals $(1, 1, \dots, 1, -1, \dots, -1)$,

$$\langle M_{0, \cdot}, w^* \rangle = a_0 + A - B = 2^m - (2^m - 1) = 1.$$

Row $(t, +)$. On this row, the flip part contributes $A - 2a_t$, and the carry part contributes

$$2 \sum_{r \leq t} b_r - B.$$

Because

$$\sum_{r \leq t} b_r = 2^m - 2^{m-t},$$

we get

$$\begin{aligned} \langle M_{(t, +), \cdot}, w^* \rangle &= a_0 + (A - 2a_t) + \left(2 \sum_{r \leq t} b_r - B \right) \\ &= 2^m - 2(2^m - 2^{m-t}) + 2(2^m - 2^{m-t}) - (2^m - 1) = 1. \end{aligned}$$

Row $(t, -)$. On this row, the flip part contributes $-A + 2a_t$, and the carry part contributes $2b_t - B$. Hence

$$\begin{aligned} \langle M_{(t, -), \cdot}, w^* \rangle &= -a_0 + (-A + 2a_t) + (2b_t - B) \\ &= -2^m + 2(2^m - 2^{m-t}) + 2 \cdot 2^{m-t} - (2^m - 1) = 1. \end{aligned}$$

So every support point has margin exactly 1 under w^* .

Finally,

$$\|w^*\|_{\infty} \geq |a_0| = |(2 - m)2^m - 1| = \Theta(m2^m) = 2^{\Omega(s)}.$$

This proves that a margin-1 separator exists, but it necessarily has exponentially large coefficient scale. That is exactly why the bound in Problems 1 and 2 depends on $\|w^*\|_1$ or $\|w^*\|_{\infty}$, not only on sparsity.

1.4 Problem 4. Experiment

The implementation is in the relative path `code/boosting_experiment.py`. The raw numerical output is in `code/boosting_summary.json`. I compared AdaBoost over the coordinate class with hinge-loss minimization under an ℓ_1 radius constraint.

1.4.1 Easy Sparse-Margin Distribution

For the easy task, I sampled $n = 400$ points in $\{-1, +1\}^{20}$ and labeled them by

$$y = \text{sign}(x_1 + x_2 + x_3).$$

This has an obvious 3-sparse separator with small coefficients.

AdaBoost reached zero training error in round 3. By round 50, it still used only 3 coordinates, its exponential loss had dropped to about

$$3.95 \times 10^{-6},$$

and its normalized margin was about

$$0.320.$$

The hinge-loss method with ℓ_1 radius 3 also fit the sample perfectly, used support size 3, and achieved normalized margin exactly

$$\frac{1}{3}.$$

So on a benign sparse problem, both methods behave well and the coordinate weak learner is already strong.

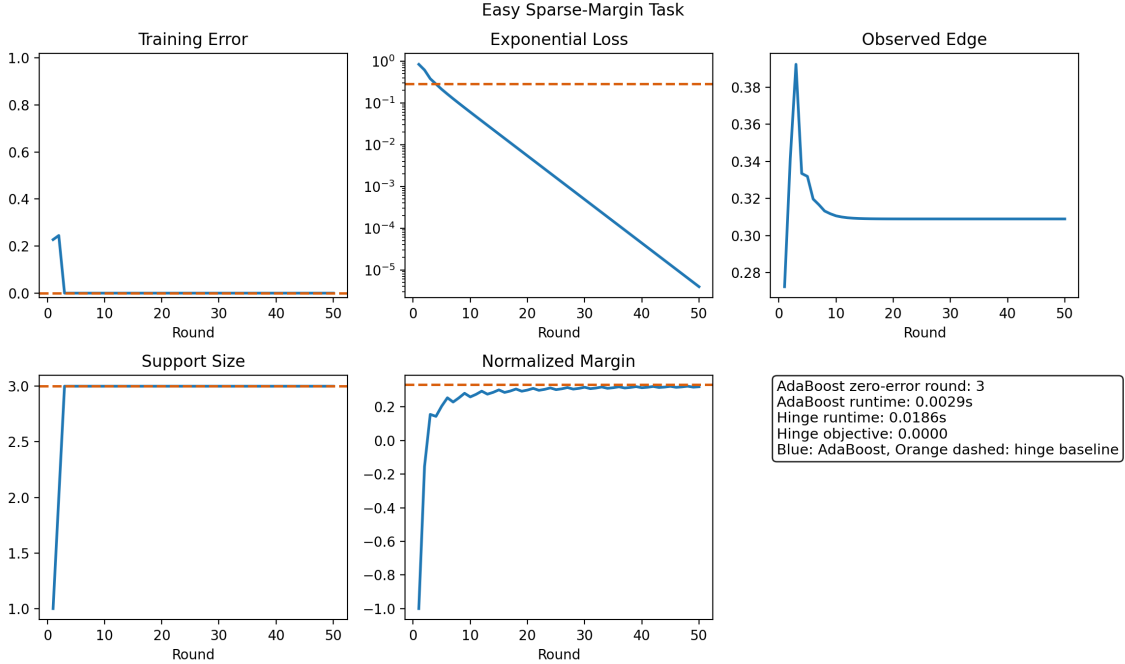


Figure 1: AdaBoost versus hinge- ℓ_1 on the easy sparse-margin task.

1.4.2 Hard Construction

For the hard task, I used the construction from Problem 3 with

$$m = 3, \quad s = 7.$$

The weighted distribution has

$$q = (1, 1, 1, 2, 2, 4, 4),$$

so the best coordinate edge is exactly

$$\frac{1}{2(2^{m+1} - 1)} = \frac{1}{30} \approx 0.0333,$$

and the script observes an initial edge of 0.0333 and a final edge of about 0.0318 after 200 rounds. AdaBoost needed 136 rounds before its training error became zero. Even after 200 rounds, the normalized margin was only about

$$0.0065,$$

which is tiny compared with the easy task. The support size quickly saturated at 7, so the real bottleneck was not support growth but the weak edge induced by poor ℓ_1 geometry.

For the convex surrogate baseline, I used hinge-loss minimization with ℓ_1 radius 7. It did *not* fit the sample: its training error stayed at 0.2, its exponential loss was about 27.3, and its normalized margin was negative. This is consistent with the theory: the hard instance has a margin-1 separator, but it requires large coefficients, and the witness from Problem 3 already has

$$\|w^*\|_\infty = 9.$$

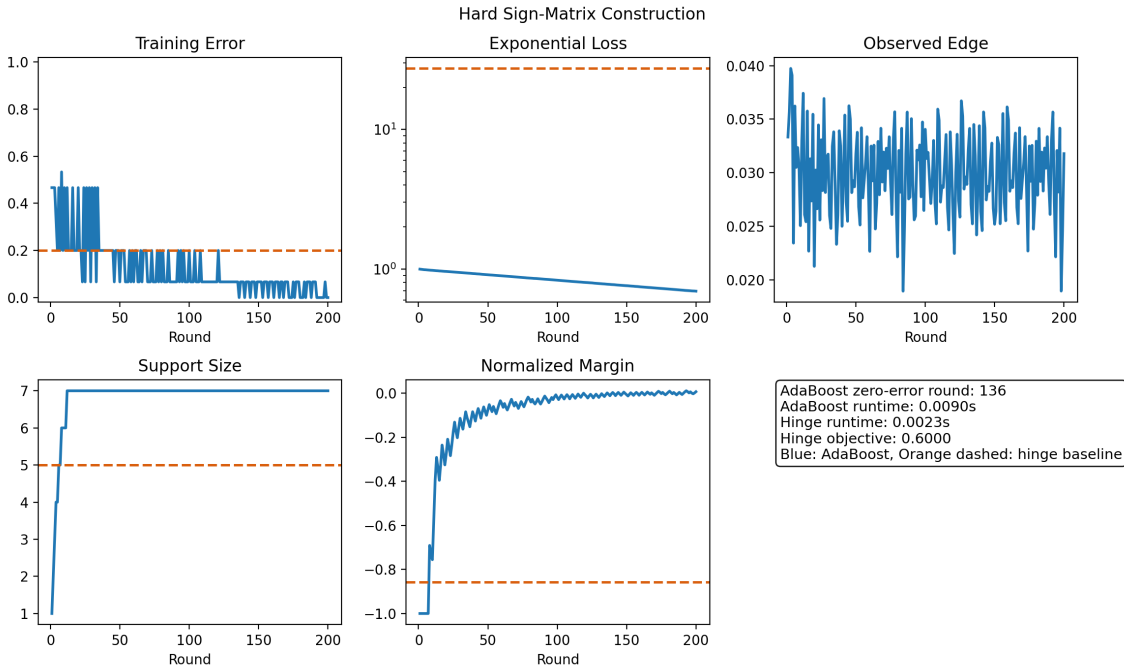


Figure 2: AdaBoost versus hinge- ℓ_1 on the hard sign-matrix construction.

1.4.3 Conclusion of the Experiment

The experiment illustrates four points.

- Support sparsity alone is not enough. The easy and hard datasets are both sparse, but only the easy one gives a large coordinate edge.
- Coefficient size matters because the weak-learning guarantee depends on $\|w^*\|_1$, not just $\|w^*\|_0$.
- AdaBoost remains computationally cheap because each round is only a coordinate search, but it can need many rounds when the edge is tiny.
- A convex surrogate with an ℓ_1 constraint can work very well when the true separator has modest coefficient size, but it can fail badly when the margin witness requires large coefficients.

2 Part B: Agnostic Halfspace Hardness via Boosting

2.1 Problem 1. A Weak Halfspace Inside an Intersection

Assume $\mathcal{D}$ is realized by

$$g(x) = \bigwedge_{r=1}^k h_r(x), \quad h_r \in \mathcal{H}_d.$$

Let

$$p := \mathbb{P}_{\mathcal{D}}[y = +1].$$

Case 1: $p \leq \frac{1}{2} - \frac{1}{2k}$. The constant classifier $h(x) \equiv -1$ is an affine halfspace, and its error is exactly p . Therefore

$$L_{\mathcal{D}}(h) = p \leq \frac{1}{2} - \frac{1}{2k} \leq \frac{1}{2} - \frac{1}{2k^2}.$$

Case 2: $p > \frac{1}{2} - \frac{1}{2k}$. Whenever $y = +1$, realizability by the intersection implies

$$h_r(x) = +1 \quad \text{for every } r \in [k].$$

So each h_r makes no error on positive examples.

Whenever $y = -1$, at least one of the k halfspaces must output -1 , otherwise the intersection would also output $+1$. Hence

$$\frac{1}{k} \sum_{r=1}^k \mathbb{P}_{\mathcal{D}}[y = -1, h_r(x) = -1] \geq \frac{1-p}{k}.$$

Therefore the average error of the h_r 's is

$$\begin{aligned} \frac{1}{k} \sum_{r=1}^k L_{\mathcal{D}}(h_r) &= \frac{1}{k} \sum_{r=1}^k \mathbb{P}_{\mathcal{D}}[y = -1, h_r(x) = +1] \\ &= (1-p) - \frac{1}{k} \sum_{r=1}^k \mathbb{P}_{\mathcal{D}}[y = -1, h_r(x) = -1] \leq (1-p) \left(1 - \frac{1}{k}\right). \end{aligned}$$

Since in this case $p \geq \frac{1}{2} - \frac{1}{2k}$,

$$(1-p) \left(1 - \frac{1}{k}\right) \leq \left(\frac{1}{2} + \frac{1}{2k}\right) \left(1 - \frac{1}{k}\right) = \frac{1}{2} - \frac{1}{2k^2}.$$

So the average halfspace error is at most $\frac{1}{2} - \frac{1}{2k^2}$, and therefore at least one affine halfspace satisfies

$$L_{\mathcal{D}}(h) \leq \frac{1}{2} - \frac{1}{2k^2}.$$

2.2 Problem 2. A Weak Learner from Proper Agnostic PAC Learning

Assume, hypothetically, that affine halfspaces are efficiently *properly* agnostically PAC learnable. I will use that learner as a subroutine on the weighted distributions generated by boosting.

Fix a distribution that is realizable by some intersection in $\mathcal{I}_{d,k}$. By Problem 1, there exists an affine halfspace h^* with

$$L(h^*) \leq \frac{1}{2} - \frac{1}{2k^2}.$$

Run the hypothetical proper agnostic learner with target excess accuracy

$$\varepsilon_{\text{agn}} := \frac{1}{4k^2}$$

and constant confidence, say $2/3$. Since the learner is agnostic, it returns an affine halfspace $\hat{h}$ such that

$$L(\hat{h}) \leq \inf_{h \in \mathcal{H}_d} L(h) + \varepsilon_{\text{agn}} \leq \left(\frac{1}{2} - \frac{1}{2k^2}\right) + \frac{1}{4k^2} = \frac{1}{2} - \frac{1}{4k^2}.$$

So this gives a weak learner with edge

$$\gamma = \frac{1}{4k^2}.$$

Why is it polynomial-time when $k(d) \leq d^c$ for fixed c ? Because then

$$\frac{1}{\varepsilon_{\text{agn}}} = 4k(d)^2 \leq 4d^{2c},$$

which is polynomial in d . Hence any runtime polynomial in d , sample size, and $1/\varepsilon_{\text{agn}}$ remains polynomial.

2.3 Problem 3. Boosting the Weak Learner

Now run AdaBoost using the weak learner from Problem 2. At every round, the reweighted distribution is still supported on the same sample, and that sample is still realized by the same intersection in $\mathcal{I}_{d,k}$. So every round has weak edge at least

$$\gamma = \frac{1}{4k^2}.$$

AdaBoost therefore satisfies the standard training-error bound

$$\widehat{L}(H_T) \leq \exp(-2\gamma^2 T) = \exp\left(-\frac{T}{8k^4}\right).$$

Thus

$$T = O(k^4 \log n)$$

rounds are enough to make the empirical training error zero on an n -point sample.

We are allowed to use the boosted-halfspace VC bound

$$\text{VCdim}(\text{boosted halfspaces with } T \text{ rounds}) = \widetilde{O}(Td).$$

Hence, once the boosted classifier is sample-consistent, the realizable VC theorem gives true error at most ε with probability at least $1 - \delta$ provided

$$n = \widetilde{O}\left(\frac{Td + \log(1/\delta)}{\varepsilon}\right).$$

Substituting $T = O(k^4 \log n)$ yields, up to logarithmic factors,

$$n = \widetilde{O}\left(\frac{dk^4 + \log(1/\delta)}{\varepsilon}\right).$$

For runtime, each of the T rounds calls the proper agnostic halfspace learner once, so the total runtime is

$$\widetilde{O}(k^4) \cdot \text{poly}(n, d)$$

when $k(d) \leq d^c$. In particular, for polynomially bounded $k(d)$ this is still polynomial.

2.4 Problem 4. The Hardness Contradiction

Combine Problems 2 and 3. If affine halfspaces were efficiently properly agnostically PAC learnable, then for every polynomially bounded

$$k(d) \leq d^c$$

we would obtain an efficient realizable learner for intersections of $k(d)$ affine halfspaces:

- Problem 2 converts the agnostic learner into a weak learner with edge $\gamma = \Theta(1/k(d)^2)$.
- Problem 3 boosts that weak learner in polynomially many rounds and keeps the final sample and runtime bounds polynomial.

So intersections of polynomially many affine halfspaces would be efficiently learnable in the realizable case.

This contradicts the stated hardness facts.

Under the uSVP hardness assumption, take

$$k(d) = d^r$$

for any fixed $r > 0$. The previous implication would give an efficient realizable learner for intersections of d^r halfspaces, contradicting the black-box hardness statement.

Under RSAT, take any polynomially bounded function with

$$k(d) = \omega(1).$$

Again the previous implication would yield an efficient realizable learner for that intersection class, contradicting the RSAT hardness statement.

Therefore the hypothetical premise must be false: efficient *proper* agnostic PAC learning of affine halfspaces would contradict these hardness assumptions. In short, the boosting reduction shows that proper agnostic halfspace learning is at least as hard as realizable learning of intersections of many halfspaces.

3 Part C: AI Usage Report

I used AI for three parts of this assignment: organizing the proof skeletons for Parts A and B, checking algebra in the hard construction from Part A.3, and drafting the experiment script for Part A.4.

The main suggestions I accepted were: reducing Problem A.1 to a correlation lower bound via $\|w^*\|_1$, using the two-case argument with $p = \mathbb{P}[y = +1]$ in Part B.1, and comparing the easy sparse-margin task with the explicit hard sign-matrix construction in the experiment.

I rejected or revised two kinds of suggestions. First, I did not leave Part A.3 as a vague existential construction; I replaced it with an explicit sign matrix and explicit coefficients. Second, I did not report the experiment only qualitatively; I verified the concrete quantities from the saved JSON output and inserted the actual observed values.

I verified the submission in two ways. For the theory parts, I checked each bound against the exact claim in the assignment and reworked the algebra where constants mattered. For the coding part, I ran `code/boosting_experiment.py` inside the `ai4lt` Conda environment, saved the figures and summary JSON, and then rebuilt the PDF with `pdflatex`.

The five most recent workflow updates in this repo that mattered here were:

- using the shared `ai4lt` Conda environment before homework work,
- standardizing on `pdflatex` for compilation,
- keeping one companion statement file per homework folder,
- storing code under the homework-local `code/` directory and figures under `figures/`,
- referencing long code by relative path in the report instead of inlining it.